scan/recursive_doubling

An AgentMPI collective scaling analysis

Abstract

This is Hillis–Steele prefix: in round k every rank sends its accumulator to $r + 2^k$ and folds anything arriving from $r - 2^k$, so after $\lceil \log_2 p \rceil$ rounds rank i holds the fold over ranks $0 \dots i$. It is the standard way to trade total work for critical-path depth, and the trade is unusually favourable in an agent setting because α dominates. Across 21 process counts from 2 to 32 the logged traffic matches the closed form $p\lceil \log_2 p \rceil - (2^{\lceil \log_2 p \rceil} - 1)$ at every size — 129 messages at $p = 32$ against the chain's 31 — and the round count matches $\lceil \log_2 p \rceil$ at every size, with no accounting gap and no power-of-two distinction. The subtracted term is the sum of the sends that have no destination, $\sum_k 2^k$, and it does something worth noticing: it exactly cancels the discontinuity that $p\lceil \log_2 p \rceil$ would otherwise have at each power of two, so this is the only family in the sweep whose message curve is convex and continuous in slope. Where the algorithm pays is total operator work, and the family view shows the cost is larger than any generated table reports. Each received message triggers one operator application for the inclusive accumulator and, from the second receive onwards, a second one for the *exclusive* accumulator, which the implementation maintains unconditionally and an inclusive scan then throws away. Summing the measured per-rank receive counts gives 227 applications at $p = 32$ where the chain performs 31 and where `cost.predict` charges 31, and 98 of the 227 — 43% — are discarded. For an exact operator that costs nothing; for an agent operator it is 98 wasted invocations, and because they run in sequence inside the same loop it also means the deepest rank performs nine applications rather than the five the round count implies. That is the most useful thing this family establishes: the algorithm is worth choosing above about $p = 4$, and its real price is roughly double what the model says.

1 What this family is

The `scan` collective under the `recursive_doubling` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.012–0.245 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

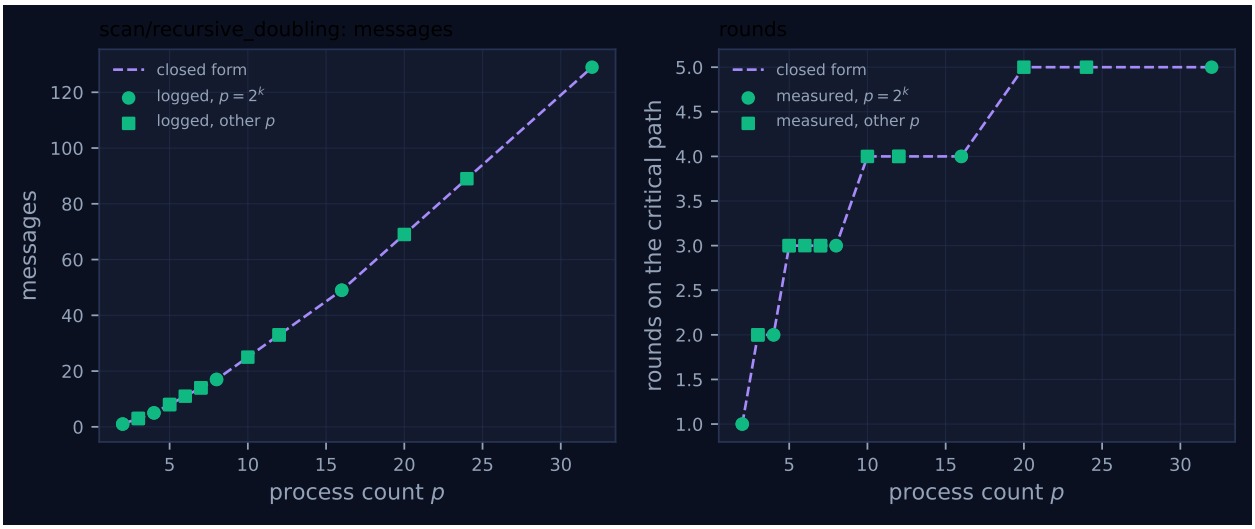

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.013	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.012	✓	✓
3	`mb-free`	3	3	3	2	2	0.013	✓	✓
3	`mb-smoke`	3	3	3	2	2	0.014	✓	✓
4	`mb-free`	5	5	5	2	2	0.017	✓	✓
4	`mb-smoke`	5	5	5	2	2	0.013	✓	✓
5	`mb-free`	8	8	8	3	3	0.017	✓	✓
5	`mb-smoke`	8	8	8	3	3	0.018	✓	✓
6	`mb-free`	11	11	11	3	3	0.032	✓	✓
7	`mb-free`	14	14	14	3	3	0.035	✓	✓
7	`mb-smoke`	14	14	14	3	3	0.058	✓	✓
8	`mb-free`	17	17	17	3	3	0.034	✓	✓
8	`mb-smoke`	17	17	17	3	3	0.028	✓	✓
10	`mb-free`	25	25	25	4	4	0.056	✓	✓
12	`mb-free`	33	33	33	4	4	0.046	✓	✓
12	`mb-smoke`	33	33	33	4	4	0.069	✓	✓
16	`mb-free`	49	49	49	4	4	0.083	✓	✓
16	`mb-smoke`	49	49	49	4	4	0.084	✓	✓
20	`mb-free`	69	69	69	5	5	0.092	✓	✓
24	`mb-free`	89	89	89	5	5	0.105	✓	✓
32	`mb-free`	129	129	129	5	5	0.245	✓	✓

4 How the algorithm scales

4.1 Where the cost comes from

A prefix reduction has p distinct answers, which is what makes it harder than a reduction. The chain produces them one at a time in $p-1$ dependent steps. Hillis–Steele produces all of them in $\lceil \log_2 p \rceil$ steps by having every rank do redundant work: after round k, rank r's accumulator covers the $\min(2^{k+1}, r+1)$ ranks ending at r, so the coverage doubles each round until it reaches back to rank 0.

The message count is where the interesting arithmetic is. Naively every rank sends in every round, giving $p\lceil \log_2 p \rceil$. But in round k the destination is $r + 2^k$, and for $r \geq p - 2^k$ that does not exist, so those ranks skip the send. The number of skipped sends in round k is exactly 2^k — and since $2^k \leq 2^{\lceil \log_2 p \rceil - 1} < p$ for every round, the count is never clamped — so the total skipped over all rounds is $\sum_{k<\lceil \log_2 p \rceil} 2^k = 2^{\lceil \log_2 p \rceil} - 1$. Hence

$$\text{messages} = p\lceil \log_2 p \rceil - (2^{\lceil \log_2 p \rceil} - 1),$$

which is the `cost.FORMULAS` entry, and its comment is right that the naive figure over-counts by nearly p. At $p = 32$: $160 - 31 = 129$. At $p = 12$: $48 - 15 = 33$. At $p = 3$: $6 - 3 = 3$. Every one of the 21 measured sizes logs exactly this number.

The traffic is markedly asymmetric, which no aggregate reports and which the per-rank send counts show directly. At $p = 8$ the eight ranks sent $3, 3, 3, 3, 2, 2, 1, 0$ messages: rank 0 sends in every round and receives nothing, rank 7 receives in every round and sends nothing, and the sequence in between is the skipping. Correspondingly the receive count at rank r is $\lfloor \log_2 r \rfloor + 1$ for $r \geq 1$ and zero for rank 0, which sums to 129 at $p = 32$ — the same figure from the other side. The algorithm is a fan-out from the low ranks and a fan-in to the high ones, simultaneously.

Fold depth is $\lceil \log_2(r+1) \rceil$ at rank r, and the measured depths confirm it: at $p = 32$ the reported values run $0, 1, 2, 2, 3, 3, 3, 3, 4, \ldots$ up to a maximum of 5.

4.2 What the figure shows: the only smooth message curve in the sweep

The rounds panel of Figure 1 is the familiar $\lceil \log_2 p \rceil$ staircase with treads at 3 for $p \in [5, 8]$, 4 for $p \in [9, 16]$ and 5 for $p \in [17, 32]$, measured points on the line at all 21 rows, both marker classes on the same treads.

The messages panel is the one worth dwelling on, because it is convex and has no cliff, and every other family with a logarithm in its formula does. `allgather/bruck`'s count is $p\lceil \log_2 p \rceil$, which jumps by 21 messages between $p = 16$ and $p = 17$ — a 33% increase for one extra rank. `allreduce/recursive_doubling`'s jumps from 109 at $p = 31$ to 160 at $p = 32$. Here the subtracted term absorbs it. Inside the bracket $(2^{k-1}, 2^k]$ the count is $pk - (2^k - 1)$, linear with slope k; crossing to $p = 2^k + 1$ the count rises by exactly $k + 1$, which is the slope of the *new* bracket. So the first differences of the message count are $1, 2, 2, 3, 3, 3, 3, 4, \ldots$ — non-decreasing, with no discontinuity anywhere. From 49 at $p = 16$ the sequence continues 54, 59, 64 at $p = 17, 18, 19$.

That is a real property and not a plotting accident, and it has a practical reading that is the opposite of Bruck's: there is no penalty for sizing a population just past a power of two. Adding one rank costs $\lceil \log_2 p \rceil + 1$ messages whether or not the addition crosses a boundary.

There are no red crosses; the self-report matches the logged traffic at all 21 rows. The algorithm has

three send paths — `sendrecv` when both a source and a destination exist, a bare `_csend` when only a destination does, a bare `_crecv` when only a source does — and each of the two sending paths carries its own `tr.sent()`. This is the same structural hazard that produced the undercount in `allreduce/recursive_doubling`, where a two-branch exchange had a `tr.sent()` on one branch only. Here both branches are instrumented, and the family view is what certifies it: 21 rows with skipped > 0 distributed across all three paths, all in agreement.

5 Powers of two and everything else

The two marker classes are indistinguishable on both panels at all 21 rows, and the reason is that Hillis–Steele has no remainder path. The guards are $0 \le r - 2^k$ and $r + 2^k < p$, and a rank for which one fails simply omits that half of the exchange. There is no participant set to renumber, no pre-phase, no post-phase, and no branch whose two sides could be instrumented inconsistently.

This is the direct comparison the sweep makes available and it is worth stating explicitly, because the same algorithm name behaves differently in the two ops. In `allreduce`, recursive doubling needs *every* rank to have a partner in every round — the result must cover all p contributions at all p ranks — so a non-power-of-two population must first be reduced to a power-of-two participant set, and that three-phase correction is where the sweep's one genuine accounting defect lives. In `scan`, rank r only ever needs data from ranks below it, so a missing partner is not an incomplete result, it is a rank whose prefix was already complete. The guard is sufficient, and there is nothing to correct.

For contrast, `allgather/recursive_doubling` has the same structural requirement as the allreduce and resolves it by refusing: the implementation substitutes Bruck at non-power-of-two p, and the sweep skips those twelve sizes, leaving that family with 9 members against this one's 21. Three ops, the same algorithm name, three different answers to the awkward case: correct by construction here, corrected with a remainder phase in `allreduce`, and abandoned in `allgather`. That the correct-by-construction one is also the only one with no defect is not a coincidence worth over-reading, but it is the pattern.

6 When to choose this algorithm

6.1 Above about four ranks, on latency and on fidelity

The comparison is against `scan/chain`, the only other implemented scan. Logged figures at matching p from `mb-free`, and modelled figures from `cost.predict` at a 1200-token payload with the repository's defaults ($\alpha = 12\,\text{s}$, $\beta^{-1} = 45$ tokens/s, $\delta = 0.05$):

	messages		rounds		modelled time (s)		worst-rank fidelity		meas. wall ratio
p	H–St.	chain	H–St.	chain	H–St.	chain	H–St.	chain	chain / H–St.
4	5	3	2	3	77.6	116.5	0.9025	0.8574	3.0×
8	17	7	3	7	116.5	271.7	0.8574	0.6983	4.2×
16	49	15	4	15	155.3	582.3	0.8145	0.4633	5.9×
24	89	23	5	23	194.1	892.9	0.7738	0.3074	8.6×
32	129	31	5	31	194.1	1203.4	0.7738	0.2039	10.5×

The crossover is at $p = 4$, where the two algorithms tie in the model at $p \le 3$ and Hillis–Steele takes a 1.5× lead at 4. That is one rank below where the implementation's default flips

(`"recursive_doubling" if p > 4 else "chain"`), which is a defensible conservatism given that $p = 4$ costs 5 messages against 3.

Above the crossover Hillis–Steele wins on both of the axes a harness usually cares about, and this is unusual enough to be worth naming. It has *lower* fold depth than the chain — 5 against 31 at $p = 32$ — so for a lossy operator it is better on latency *and* better on quality, the same way a binomial reduce dominates a chain reduce. The chain's quality problem is also unevenly distributed in a way the aggregate hides: its measured per-rank fold depths are exactly $0, 1, 2, \ldots, 31$, so under $F(d) = (1 - \delta)^d$ a chain scan degrades geometrically along the rank axis and the last rank retains 20% where the first retains 100%. Hillis–Steele's depths are $\lceil \log_2(r + 1) \rceil$, so the worst rank retains 77% and the spread across ranks is small. For the use case the `scan` docstring names — a translation in which chapter i must respect the register established in chapters $0 \ldots i - 1$ — this is the whole argument, and it is not about speed.

The measured wall-clock ratios in the last column agree in direction and overstate in magnitude: 10.5× at $p = 32$ against a 6.2× round-count ratio. The excess is poll backoff, which penalises a 31-deep critical path far more than a 5-deep one, so the seconds should be read as corroboration of the sign and not as a measurement of the factor.

6.2 What it costs, which is more than the model says

Hillis–Steele's price is total operator work, and here the family view produces something the generated tables do not contain.

The message count is a lower bound on the application count, not equal to it. Reading the loop: each received message triggers `op.fn` once for the inclusive accumulator, and — from the second receive onwards — a second time for the *exclusive* accumulator (`excl = recvd["acc"] if not got_any else op.fn(recvd["acc"], excl, ...)`). The exclusive accumulator is maintained unconditionally, and `out = excl if exclusive else inclusive` discards it whenever the caller asked for an inclusive scan, which every run in this family did.

Counting from the measured per-rank receive totals:

p	receives (= messages)	inclusive folds	exclusive folds	total applications	chain's
8	17	17	10	27	7
16	49	49	34	83	15
32	129	129	98	227	31

So at $p = 32$ the algorithm performs 227 operator applications against the chain's 31 — a factor of 7.3, not the 1.9× that `cost.predict`'s price column reports, because `predict` charges the operator-output term as $(p - 1) \cdot$ `op_cost_tokens` for every algorithm regardless of how many applications it performs. And 98 of the 227, 43%, are applications whose result is discarded. At the model's default 1200 output tokens and \$15/Mtok that is about \$1.76 of pure waste per scan at $p = 32$, against a total modelled price of \$1.49 for the whole collective — which is to say the model's price for this algorithm is out by more than a factor of two, in the cheap direction.

The wasted applications are also on the critical path, because they execute in sequence inside the same loop as the useful ones. Rank 31 at $p = 32$ receives five messages and therefore performs five inclusive folds and four exclusive ones: nine applications, not the five its fold depth and the round count imply. With a semantic operator at $\alpha = 12\,\text{s}$ that is the difference between about 194 s and

about 348 s. So the round count understates this algorithm's agent-latency critical path by nearly a factor of two, and it would not if the exclusive fold were guarded by `if exclusive`.

I would call the unconditional exclusive fold a genuine defect rather than a modelling nicety, though a much less serious one than the accounting bug in `allreduce/recursive_doubling`: it is invisible for exact operators, which is every operator in the archive, it produces correct results, and the fix is a one-line guard. What it does is silently double the cost and the depth of the one collective whose entire justification is minimising depth.

6.3 Where the chain still wins

Two places, and only two. When the operator is declared `Associativity.NONE` the tree is refused outright and the chain is the only legal algorithm; no crossover applies. And when operator applications are the binding cost rather than latency — a very expensive operator, a population small enough that $p - 1$ sequential invocations are tolerable — the chain's $p - 1$ applications is the floor and Hillis–Steele's 227 at $p = 32$ is 7.3 times it. Every other axis favours Hillis–Steele from $p = 4$ upward.

7 Threats to this reading

No agents in any of these runs, and the argument in §3.2 is where that bites hardest. All 21 members record `executors: {none: p}`, zero agent calls, zero tokens in and out, \$0.0000, `total_busy_s: 0` and `idle_fraction: 1.0`; the operator is `SUM` in every run, and `SUM` is `Associativity.EXACT`, so 227 additions cost nothing and lose nothing. The application counts, the \$1.76 of waste and the 348 s critical path are all consequences of `cost.py`'s default parameters applied to a count I derived from the code; nothing in the traces measures them, because an exact operator leaves no `agent.call` events to count. A single run of this algorithm with a real executor would settle the whole question by counting invocations, and none exists.

The application count is derived from measured receive counts plus six lines of source. The receive totals — 17, 49, 129 — are measured directly from `msg.recv` events. The step from receives to applications is a reading of `algorithms.scan`: one `op.fn` per receive for `inclusive`, one more per receive after the first for `excl`. That reading is straightforward but it is a reading, and if the exclusive branch were short-circuited somewhere I have not seen, the 98 would be zero and §3.2 would collapse. The trace cannot distinguish the two, which is exactly why I have labelled it an inference.

Wall times measure a polling schedule, not the protocol. A blocked receive sleeps `POLL_INTERVAL` = 10 ms growing by 1.4× to `MAX_POLL_INTERVAL` = 250 ms, so an algorithm's measured duration is roughly its critical-path depth times a growing discovery delay. This family is the shallow one and so is flattered: 0.245 s at $p = 32$ for 129 messages, against `scan/chain`'s 2.252 s for 31. The campaign repeats show the noise floor — 0.056 s and 0.069 s at $p = 12$, 0.035 s and 0.058 s at $p = 7$ — so the seconds are good to about a factor of two and support only the large structural gaps.

The token volumes are framing. Every message in this family is exactly 11 wire tokens, at every p, because the payload is `{"acc": 1, "depth": 0}` with an integer accumulator. Total wire volume is therefore 11× the message count, and the closed form's $(p\lceil\log_2 p\rceil - 2^{\lceil\log_2 p\rceil} + 1) \cdot n$ volume term is untested by a sweep in which $n = 1$. Nothing above rests on volume.

The fidelity comparison uses a δ nobody measured here. The per-rank fold depths $\lceil \log_2(r+1) \rceil$ are measured and are the right input, but $\delta = 0.05$ is `CostParams`'s default. All the retention percentages are "what the model says". `bench_fidelity` is where a measured δ would come from, and it measures `reduce`, not `scan`, so even a calibrated δ would be imported rather than native.

Every run passed `algorithm="recursive_doubling"` explicitly and `exclusive=False` implicitly. So the family measures the inclusive scan only, which is the case in which the exclusive fold is wasted; an exclusive scan would use all 227 applications productively and §3.2's waste figure would be zero. The archive contains no `coll.exscan` records at all, so the exclusive variant of either algorithm is entirely unmeasured, and the claim that the guard is safe rests on reading `out = excl if exclusive else inclusive` rather than on a test.